

Supapixel-Guided Iterative Prompting (SGIP)

Method Specification (aligned with code)

1. Inputs / Hyper-parameters

Required inputs

- Image: I
- Resize long side: L (default: 260). Short side scaled proportionally.
- SLIC:
 - number of superpixels: K (default: 36)
 - compactness: c
 - color space: RGB
 - sigma: σ
 - min size factor: $f_{\min}$
 - max size factor: $f_{\max}$

Core hyper-parameters (tune first)

- Center window range: $[l, h] \subset (0, 1)$ (`center_range`)
- Init prompt counts: positives n_+ , negatives m_-
- Foreground update threshold: τ (`threshold.value`)
- Subset-point filtering: `use_subset_points`
- Convex-hull refinement: `use_convex_hull`
- Mask pool de-dup: `deduplicate_mask_pool`
- Non-convex threshold: τ_{convex} (`convex_hull_threshold`)
- Positive-point prune distance: $d_{\min}$
- Mask de-dup IoU threshold: τ_{iou} (`mask_pool_iou_threshold`)

Secondary hyper-parameters (usually keep default)

- Candidate pool size: q (`candidate_top_k`)
- Max iterations: T
- Initial color mode: `initial_color_mode`

2. Notation

- Resized image: $I' = \text{ResizeLongSide}(I, L)$
- Superpixel label map: $S \in \{1, \dots, K\}^{H' \times W'}$
- Superpixel pixel set: $\Omega_j = \{x \mid S(x) = j\}$
- Superpixel centroid (float): μ_j

- Superpixel intensity proxy: g_j (mean intensity or equivalent, depending on `initial_color_mode`)
- Prompt point set: $P = \{(u_i, y_i)\}, y_i \in \{+1, -1\}$
- Mask prompt: $M^{\text{prompt}} \in \{0, 1\}^{H' \times W'}$

SAM2 inference (given prompts):

$$(\ell, M, s) \leftarrow \text{SAM2}(I', M^{\text{prompt}}, P)$$

- ℓ : logits map
- M : output mask
- s : SAM2 output score (quality score for mask)

Logits thresholding:

$$\hat{M} = \mathbb{K}[\ell > 0]$$

Probability map:

$$p(x) = \sigma(\ell(x))$$

Soft vote (mean logits in superpixel):

$$v_j = \frac{1}{|\Omega_j|} \sum_{x \in \Omega_j} \ell(x)$$

4. Algorithm

Main Flow (Module-Level)

1. **Input Image:** raw input I .
2. **Preprocess and Superpixels:** resize and generate superpixels with derived statistics.
3. **Initialize Prompts:** build initial positive/negative point prompts and empty mask prompt.
4. **Iterative Prompting with SAM2:** refine prompts and accumulate candidate masks.
5. **Final Mask Selection:** select the best mask from the candidate set.
6. **Output Mask:** final prediction M^* .

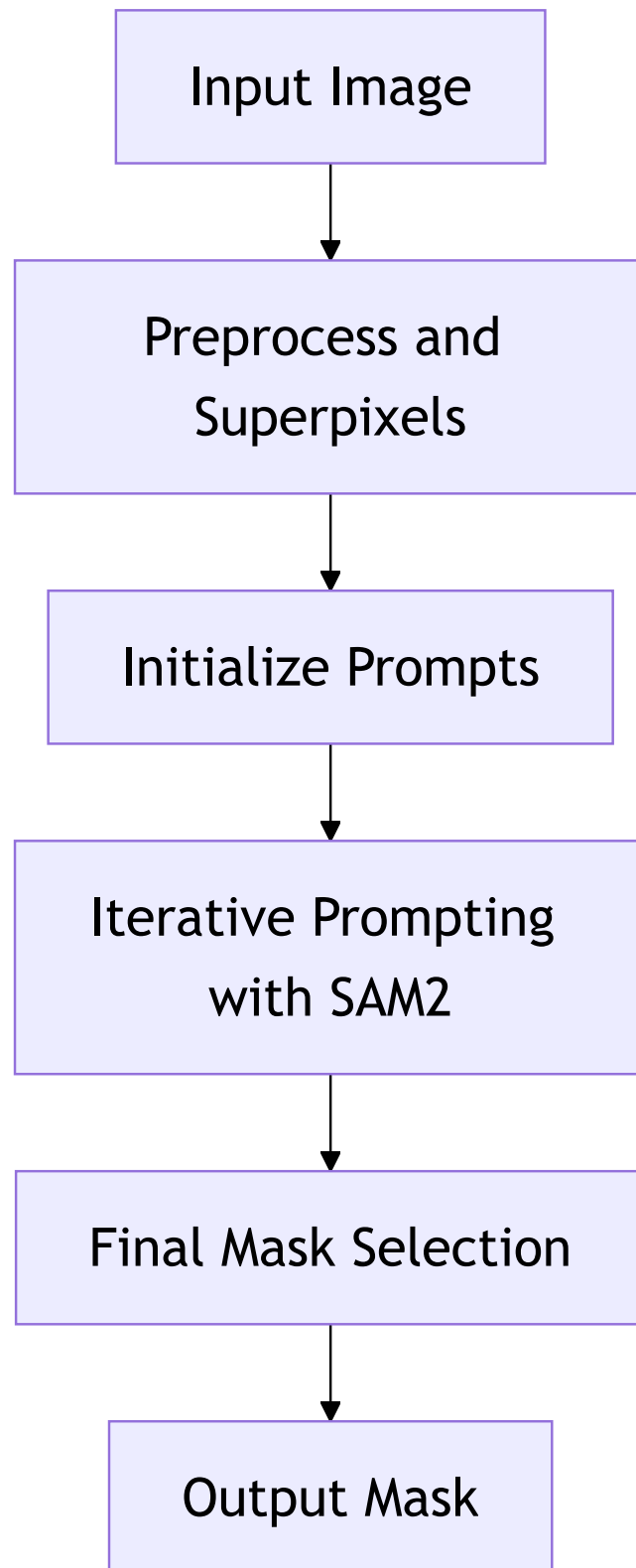


Figure 1: Main SGIP flow

Step 1 — Preprocess & Superpixels

1. **Input Image:** start from I .
2. **Resize Long Side:** $I' \leftarrow \text{ResizeLongSide}(I, L)$.
3. **SLIC Superpixels:** $S \leftarrow \text{SLIC}(I', K, c)$ in RGB space.
4. **Compute Superpixel Sets Ω_j :** $\Omega_j \leftarrow \{x \mid S(x) = j\}$.
5. **Compute Centroids μ_j :** $\mu_j \leftarrow c(\Omega_j)$ (deterministic placement).
6. **Compute Intensity Proxy g_j :** $g_j \leftarrow \frac{1}{|\Omega_j|} \sum_{x \in \Omega_j} \text{Gray}(I'(x))$.

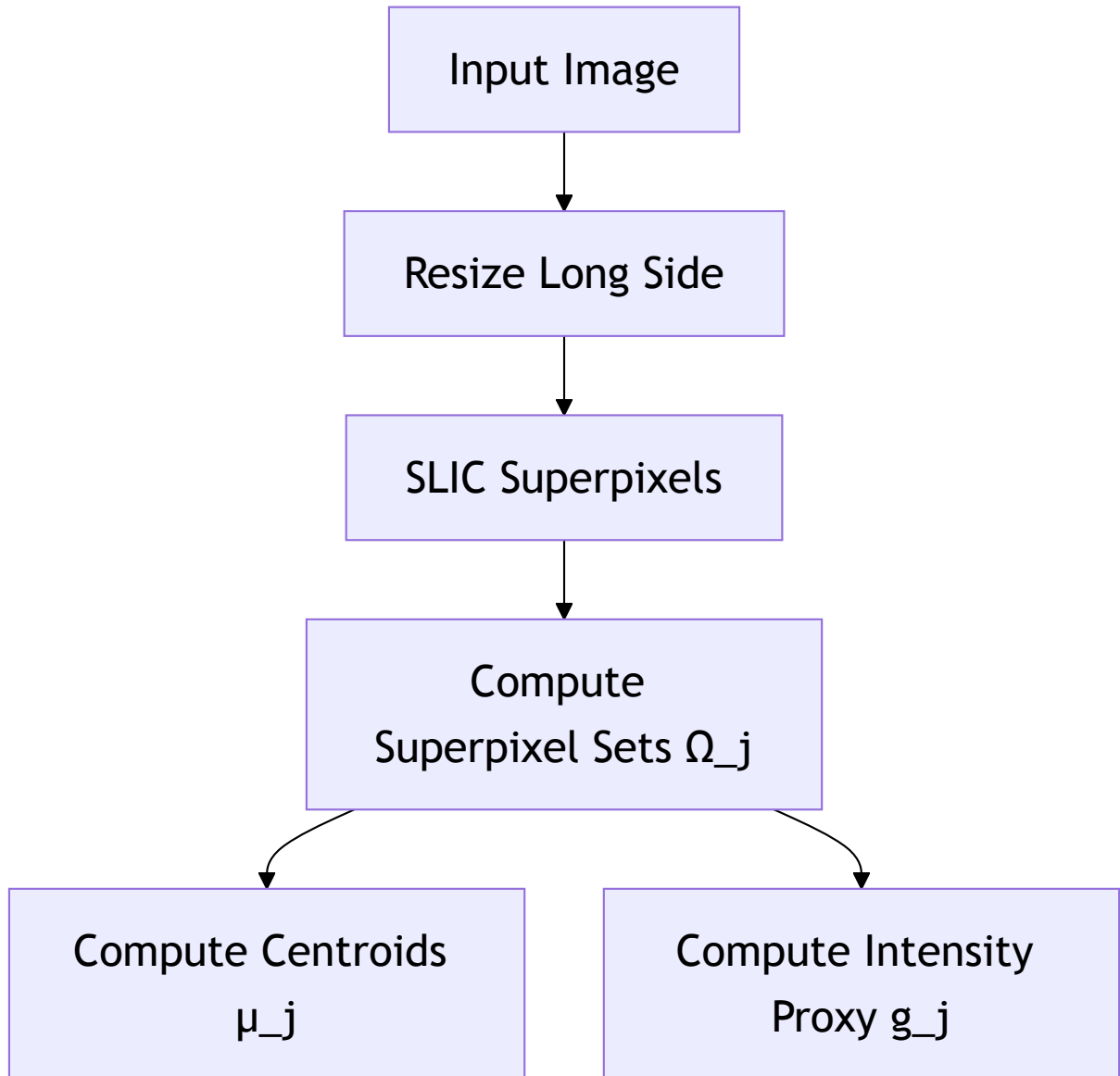


Figure 2: Preprocess & superpixels

Step 2 — Initialize Prompts

1. **Superpixels μ_j and g_j :** use centroids and intensity proxies from Step 1.
2. **Select Center-Window Superpixels:** $\mathcal{C} \leftarrow \{j \mid \mu_j^{\text{norm}} \in [\alpha, 1 - \alpha]^2\}$.
3. **Sort by Intensity Proxy g_j (center):** sort $\mathcal{C}$ by g_j descending.
4. **Select Top n_+ as Positive Seeds:** $\mathcal{J}_+ \leftarrow$ first n_+ indices.
5. **Add Positive Prompts:** $P \leftarrow \{(\mu_j, +1) \mid j \in \mathcal{J}_+\}$.
6. **Select Boundary Superpixels:** $\mathcal{B} \leftarrow \{j \mid \Omega_j \cap \partial\Omega \neq \emptyset\}$.
7. **Sort by Intensity Proxy g_j (boundary):** sort $\mathcal{B}$ by g_j descending.
8. **Select Top m_- as Negative Seeds:** $\mathcal{J}_- \leftarrow$ first m_- indices.
9. **Add Negative Prompts:** $P \leftarrow P \cup \{(\mu_j, -1) \mid j \in \mathcal{J}_-\}$.
10. **Initialize Prompt Set P :** maintain PosIdx and NegIdx.
11. **Initialize Mask Prompt Empty:** $M^{\text{prompt}} \leftarrow \mathbf{0}$.
12. **Initialize Candidate Pool Q Empty:** $\mathcal{Q} \leftarrow []$.

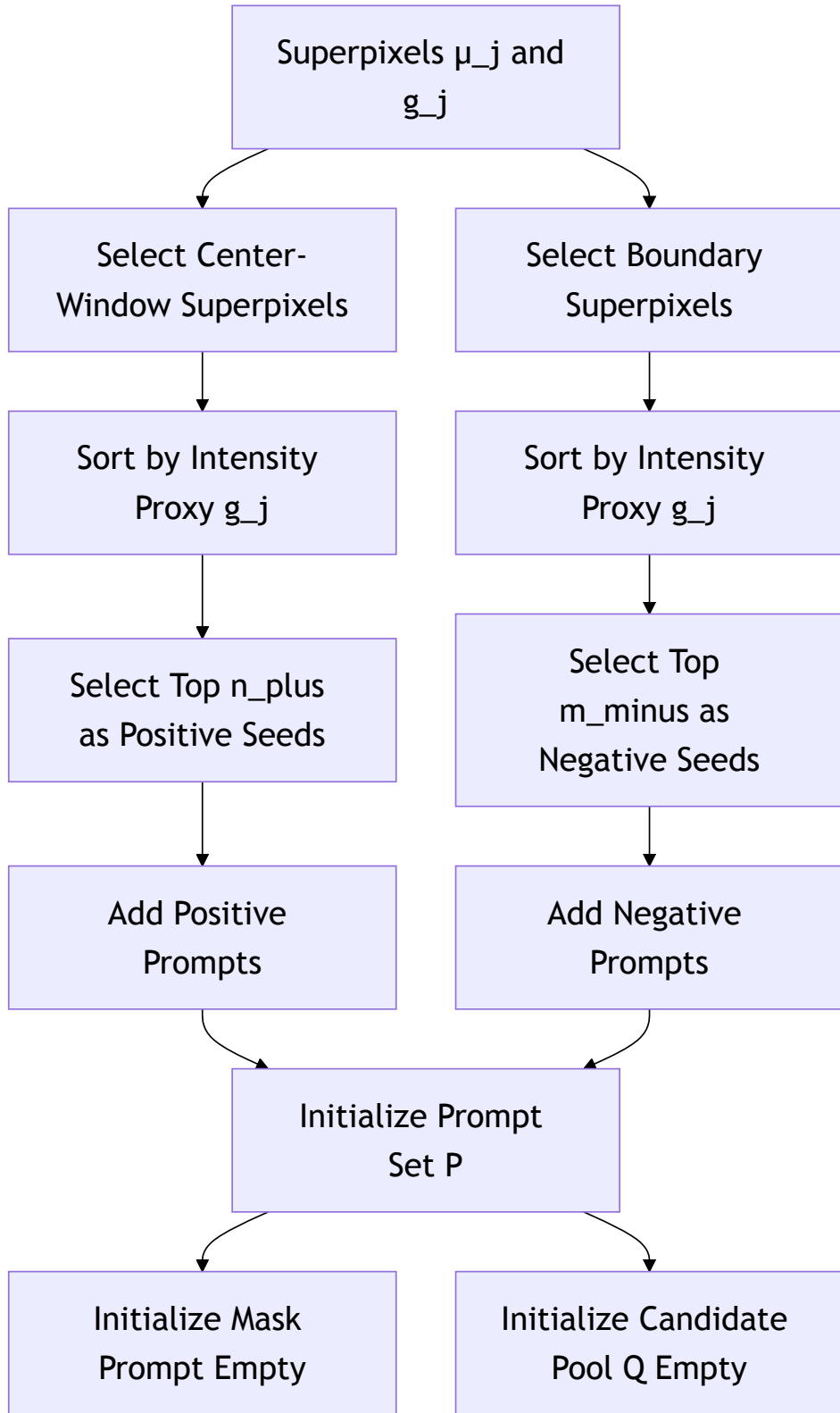


Figure 3: Initialize prompts

Step 3 — Iterative Prompting with Candidate Evaluation

For $t = 1, 2, \dots, T$:

1. **Current Prompts and Mask:** use the current P and M^{prompt} .
2. **Generate Superpixel Scores:**

$$(\ell_t, M_t, s_t) \leftarrow \text{SAM2}(I', M^{\text{prompt}}, P), \quad p_t \leftarrow \sigma(\ell_t)$$

$$v_j = \frac{1}{|\Omega_j|} \sum_{x \in \Omega_j} \ell_t(x), \quad j \notin (\text{PosIdx} \cup \text{NegIdx})$$

3. **Select and Add Best Candidate:** select top- q candidates, evaluate each with an extra positive point $(\mu_j, +1)$, choose j^* by highest score, and update P and PosIdx .
4. **Point placement note:** when adding a prompt for a region R , use its centroid $c(R)$ if $c(R) \in R$; otherwise use the nearest pixel in R to $c(R)$. This avoids centroid-outside-region issues.
5. **Optional Convex Hull / Pruning:** apply selective convex hull and/or prune close positives.
6. **Update Mask Prompt + Append to Pool:**

$$M^{\text{prompt}} \leftarrow \left(\bigcup_{(u, +1) \in P} \Omega_{S(u)} \right), \quad \mathcal{Q}.\text{append}((M_t, s_t))$$

7. **More Valid Candidates?** stop if none remain.

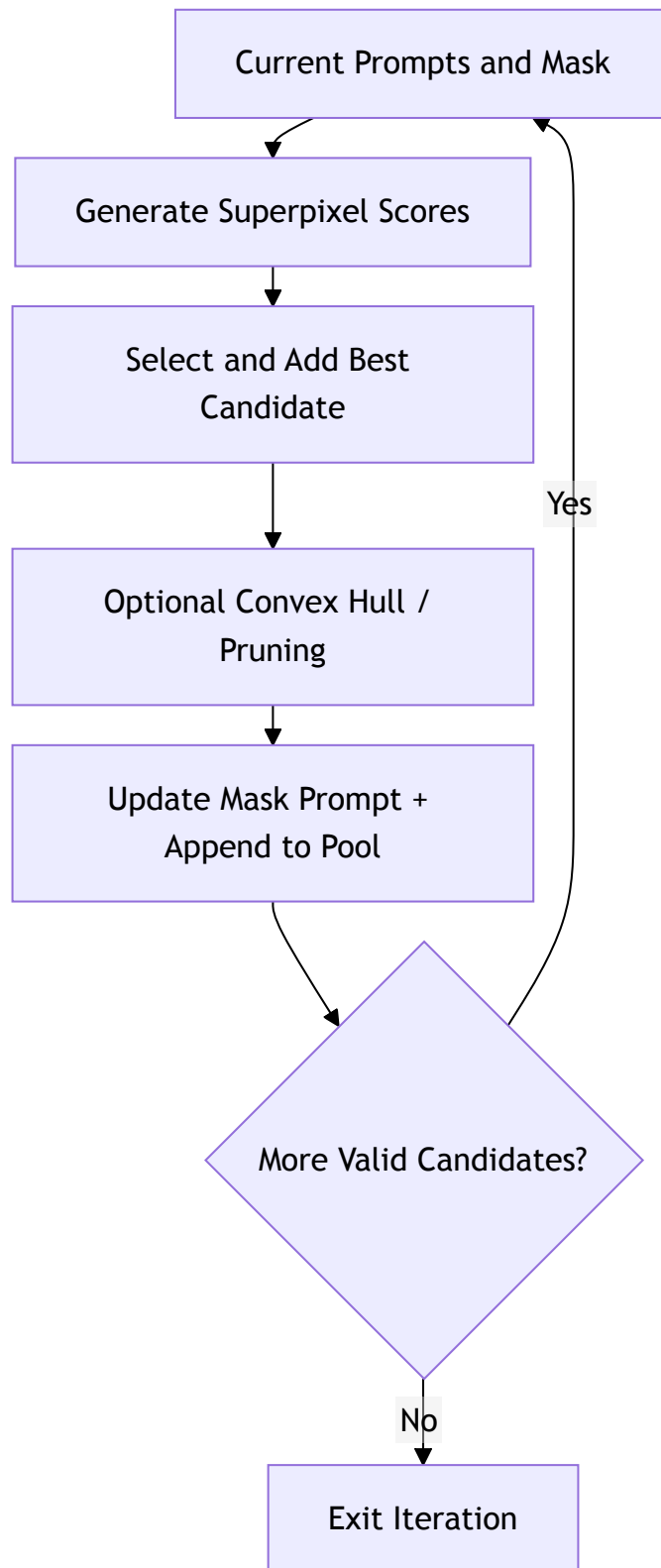


Figure 4: Iterative prompting with candidate evaluation

Step 3.1 — Candidate Pool Evaluation (`evaluate_candidates`)

1. **Valid candidate set U :** unlabeled, non-boundary superpixels.
2. **Compute v_j :** score each candidate by mean logit in its superpixel.
3. **Select Top- q candidates K :** $q = \text{candidate_top_k}$.
4. **Evaluate each $j \in K$:** run SAM2 with $(\mu_j, +1)$ and record $s^{(j)}$.
5. **Pick best candidate:** choose j^* with highest $s^{(j)}$ and keep its mask M_t .

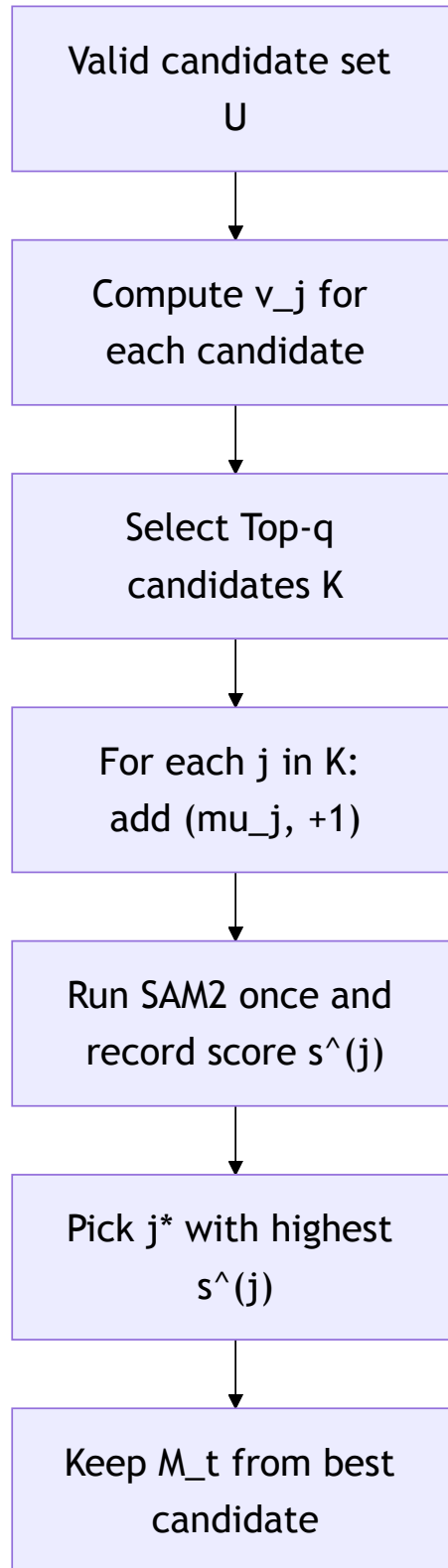


Figure 5: Candidate pool evaluation

Step 4 — Final Mask Selection

Let candidates be $\{(M_i, s_i)\}_{i=1}^N$ from $\mathcal{Q}$.

1. **IoU De-duplication:** if $\text{IoU}(M_a, M_b) > \tau_{\text{iou}}$, keep the higher-score mask.
2. **Remaining Masks ≥ 3 ?** if not, return $M^\star = \arg \max_i s_i$.
3. **Cluster Masks by Area:** $k = 3$ with centers at $\min(a)$, $\text{median}(a)$, and $\max(a)$.
4. **Select Middle Cluster:** choose C_{mid} by centroid area.
5. **Pick Max-Score Mask:** $M^\star = \arg \max_{M_i \in C_{\text{mid}}} s_i$.

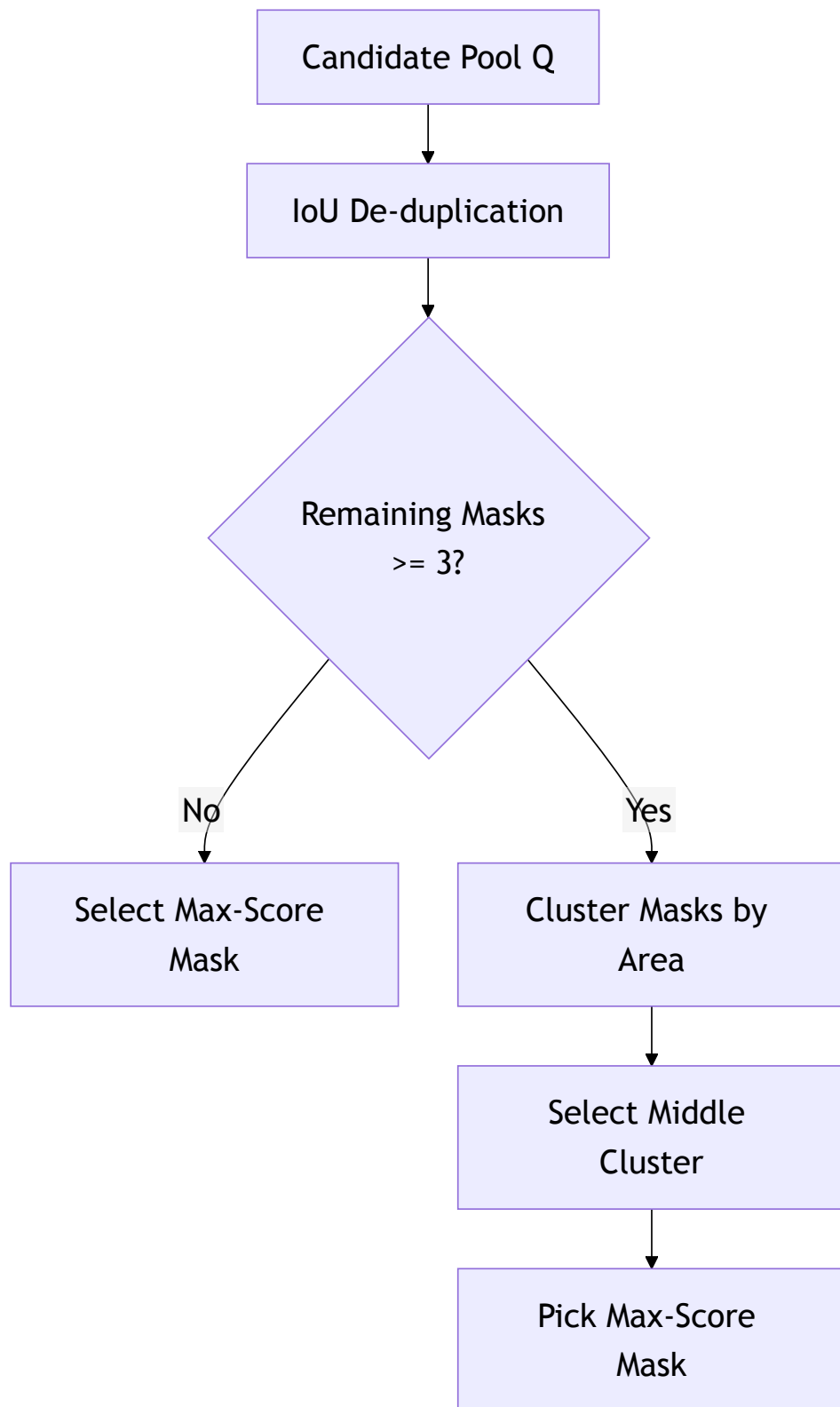


Figure 6: Final mask selection

Output

Final mask M^* .